Engenharia de Software – Aula 26

Sprint 04: Diagrama de Sequência e Orquestração do Negócio

Prof. Me. Reginaldo Pereira Fernandes

Instituto Federal de Educação, Ciência e Tecnologia do Ceará – Campus Tauá
Técnico em Informática para Internet (TSII.210)

Semana 2 – Encontro 3 (100 min)

Agenda do Encontro 3 (100 Minutos)

1. Modelagem Dinâmica: Diagrama de Sequência
2. Diagrama de Atividades e Regras de Negócio
3. Implementação: Calculadora de Desconto
4. Implementação: Integração com o Pedido
5. Implementação: CheckoutService Orquestrador
6. Rito Ágil e Próximos Passos

Por que Precisamos da Visão Dinâmica?

- **O que já temos até a Aula 25:**
  - Entidade `Cupom` com regras de negócio em Python.
  - Tabela `cupons` e `CupomRepository` persistindo no SQLite.
- **A Pergunta que o Diagrama de Classes Não Responde:**
  - *"Em que ordem temporal os objetos conversam durante a compra de um livro com cupom?"*
  - *"Quem deve consultar o repositório? Quem deve debitar o estoque? Quem deve atualizar o total?"*
- **Diagrama de Sequência (UML 2 – Cap. 5):**
  - O padrão industrial para modelar o fluxo de execução síncrono e transacional entre componentes.

Diagrama de Sequência: Fluxo do Checkout com Cupom

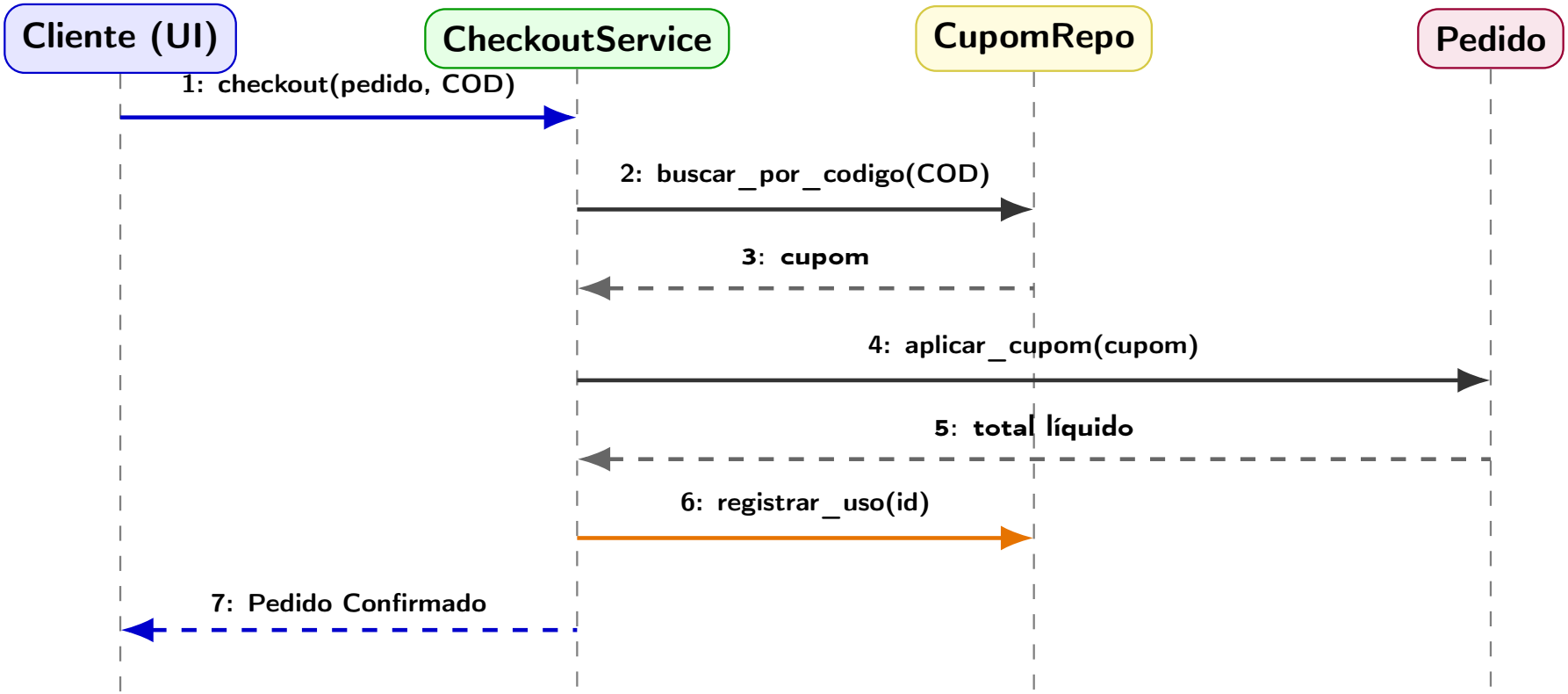
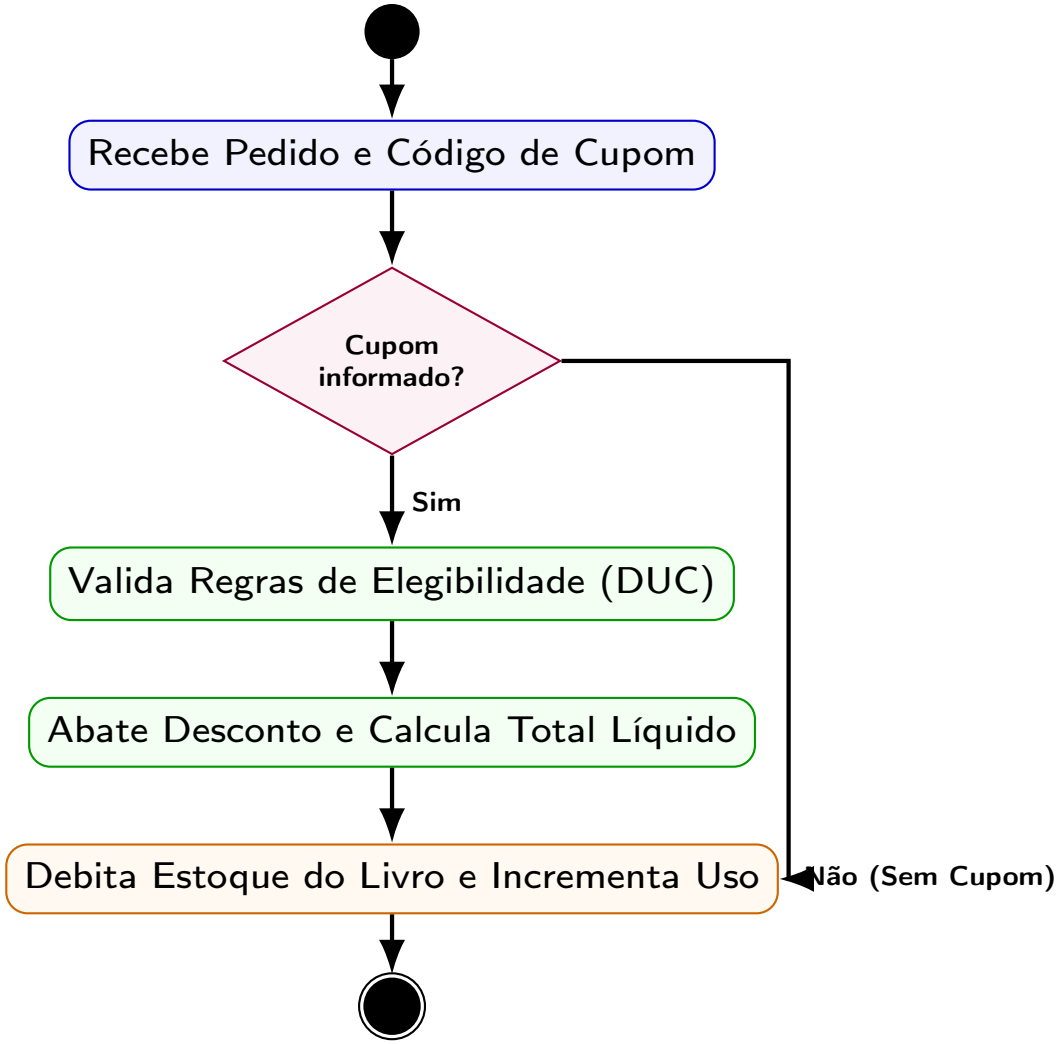


Diagrama de Atividades: Ciclo de Decisão no Checkout



Mão na Massa: CalculadoraDescontoService

```python
class CalculadoraDescontoService:
    """Servico de dominio puro: calculo matematico de descontos."""

    @staticmethod
    def calcular(subtotal: float, tipo_desconto: TipoDesconto, valor_desconto
        : float) -> float:
        if subtotal <= 0:
            return 0.0

        if tipo_desconto == TipoDesconto.PERCENTUAL:
            desconto = subtotal * (valor_desconto / 100.0)
        elif tipo_desconto == TipoDesconto.VALOR_FIXO:
            desconto = min(valor_desconto, subtotal)
        else:
            raise ValueError(f"Tipo de desconto desconhecido: {tipo_desconto}
                ")

        return round(desconto, 2)
```

Por que um Serviço Isolado?

Permite testar e reutilizar as fórmulas matemáticas de desconto sem depender de banco de dados ou da interface!

Mão na Massa: Integrando o Cupom ao Pedido

```python
class Pedido:
    def __init__(self, cliente: Cliente, id: Optional[int] = None):
        self.id = id
        self.cliente = cliente
        self.itens: List[ItemPedido] = []
        self.cupom: Optional[Cupom] = None
        self.desconto_aplicado: float = 0.0
        self.status: str = "CRIADO"

    @property
    def subtotal(self) -> float:
        return round(sum(item.subtotal for item in self.itens), 2)

    def aplicar_cupom(self, cupom: Cupom) -> float:
        desconto = cupom.calcular_desconto(self.subtotal)
        self.cupom = cupom
        self.desconto_aplicado = desconto
        return self.desconto_aplicado

    @property
    def total(self) -> float:
        return max(round(self.subtotal - self.desconto_aplicado, 2), 0.0)
```

Mão na Massa: CheckoutService Transacional

```python
class CheckoutService:
    def __init__(self, cupom_repo: CupomRepository, livro_repo:
        LivroRepository, pedido_repo: PedidoRepository):
        self.cupom_repo = cupom_repo
        self.livro_repo = livro_repo
        self.pedido_repo = pedido_repo

    def processar_checkout(self, pedido: Pedido, codigo_cupom: Optional[str]
        = None) -> Pedido:
        # 1. Se informou cupom, busca e aplica
        if codigo_cupom:
            cupom = self.cupom_repo.buscar_por_codigo(codigo_cupom)
            if not cupom:
                raise CupomInexistenteError(codigo_cupom)
            pedido.aplicar_cupom(cupom)

        # 2. Debita estoque dos livros adquiridos
        for item in pedido.itens:
            self.livro_repo.atualizar_estoque(item.livro.id, -item.quantidade
                )

        # 3. Registra uso do cupom se aplicado
        if pedido.cupom:
            self.cupom_repo.registrar_uso(pedido.cupom.id)

        # 4. Persiste o pedido e finaliza status
        pedido.status = "PAGO"
        return self.pedido_repo.salvar(pedido)
```

Rito Ágil: Registro da Daily 3

Anotação para o Relatório da Sprint no `guia_desenvolvedor.md`

- **O que realizei hoje?**
  - Interpretei o Diagrama de Sequência UML do fluxo de checkout.
  - Implementei a `CalculadoraDescontoService`.
  - Integrei o agregado `Pedido` ao cupom com recálculo automático do total líquido.
  - Implementei o orquestrador `CheckoutService` garantindo baixa de estoque e registro de uso.
- **O que farei no Encontro 4 (Aula 27)?**
  - Construir a suíte de testes unitários e de integração com **Pytest**.
  - Demonstrar o fluxo interativo no CLI e submeter o Pull Request final.
- **Impedimentos?** Nenhum impedimento.

Síntese do Encontro 3

- **Sistema Totalmente Integrado:**
  - A modelagem dinâmica se materializou em serviços coordenados.
  - O cálculo de desconto e a baixa de estoque funcionam de ponta a ponta.
- **Para o Encontro 4 (Aula 27 - Fechamento da Sprint):**
  - Validação de Qualidade: **Pytest** 100% verde.
  - Demonstração CLI para a liderança.
  - Submissão do Pull Request e Retrospectiva da Sprint.

Excelente trabalho! O Módulo de Cupons está quase pronto para entrega!